Technical Analysis: Real-Time Hand Gesture Recognition Framework

1. Problem Statement: The Evolution of Human-Computer Interaction (HCI)

Modern Human-Computer Interaction (HCI) is defined by the transition from **pixel-level processing** to **region-level understanding**. This shift is characterized as a problem of image segmentation, where an image I on domain Ω is partitioned into regions $\{R_1, R_2, ..., R_n\}$.

- The union of all regions covers the entire domain: ($\cup R_i = \Omega$).
- The intersection of distinct regions is empty: ($R_i \cap R_j = \varnothing$ for $i \neq j$).

This formalism motivates the pipeline design; in practice, the classical pipeline uses landmark-based region extraction and the deep-learning pipeline uses bounding-box detection. This formal partitioning allows a system to move beyond raw intensity values to semantic coherence, identifying exactly which pixels belong to the foreground interface (the hand) versus the background environment.

The Significance of Gesture Recognition Utilizing:
HaGRID (Hand Gesture Recognition Image Dataset), a large sample source of **30k images**, yielding **~10k usable samples** after extracting the 6 target classes. This large sample source acts as the foundational corpus, enabling touchless interfaces across diverse sectors.
The **primary mission** of this project is rooted in **assistive technologies**, aiming to provide a vital control modality for individuals with motor impairments by creating a hand gesture detector for computer usage as a dedicated alternative for non-verbal users. This core objective is further supported by applications in **industrial robotics**, where it facilitates human-computer collaboration via intuitive commands, and **consumer electronics**, enabling high-precision augmented reality and virtual background separation.
Technical Challenges
The core difficulty in gesture recognition lies in maintaining robustness against:

- **Intra-class Variation:** Significant appearance differences within the same gesture class due to individual hand morphology.
- **Illumination Changes:** Shadows and highlights that alter pixel values, requiring algorithms to separate reflectance from illumination.
- **Occlusion and Ambiguous Boundaries:** Self-occlusion of fingers and the difficulty of defining precise edges between the hand and complex backgrounds.This framework addresses these challenges by benchmarking two distinct paradigms: a **Classical Pipeline** based on geometric engineering and a **Deep Learning Pipeline** leveraging neural architectures.

2. Methodology: Dual-Pipeline Architecture and Algorithm Design

The architecture is bifurcated to evaluate the strategic trade-off between the **explainability** of handcrafted geometric features and the **raw performance** of learned deep representations.

2.1 The Classical Pipeline (Geometric Engineering)

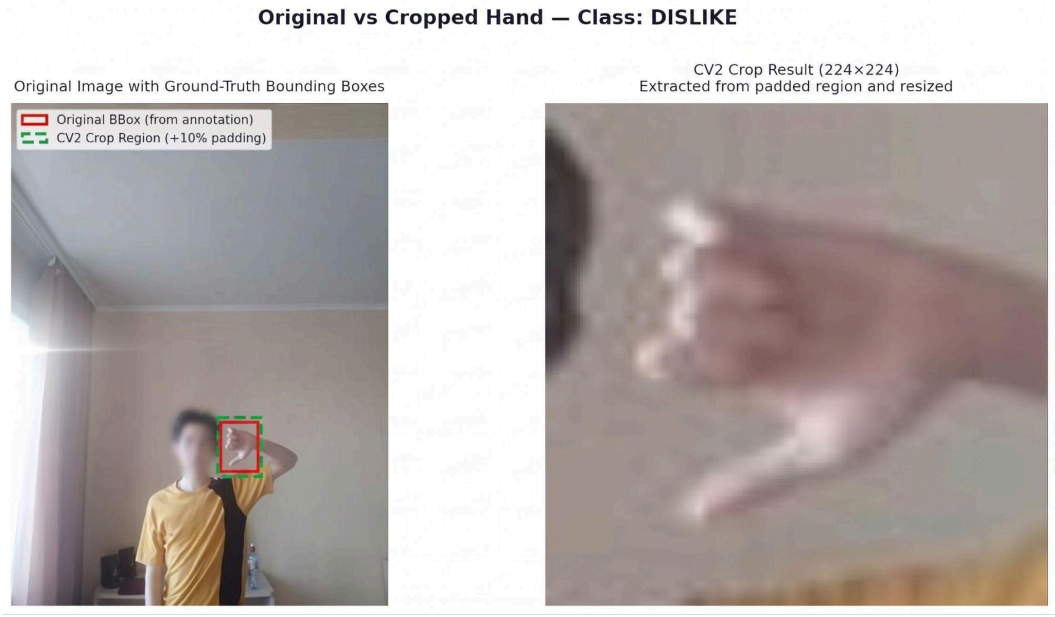
The classical approach utilizes the **MediaPipe Hand Landmarker** (Tasks API) for 21-landmark localization in a single pass. These normalized 3D landmarks are then transformed into a feature vector comprising **exactly 54 elements** .**Feature Extraction Analysis:**

- **Wrist Distances (20 features):** Euclidean distances from the wrist to every other joint.
- **Finger Segments (15 features):** Consecutive joint-to-joint distances across the five fingers.
- **PIP Angles (4 features):** Interior angles at the proximal interphalangeal joints, calculated via the Cosine Law.
- **Tip-to-Tip Distances (10 features):** Pairwise distances between the five fingertips to capture hand "openness."
- **Finger Ratios (5 features):** The ratio of tip-to-base distance over total bone length, where 1.0 indicates a straight finger and ~0.3 indicates a curled posture. **Normalization by palm size** (distance from wrist to middle finger MCP) is applied to all distance features to ensure **scale-invariance** . The classification is performed by a **Support Vector Machine (SVM)** utilizing a **Radial Basis Function (RBF) kernel** and **StandardScaler** to ensure zero-center features and unit variance.

2.2 The Deep Learning Pipeline (Neural Architecture)

This pipeline follows a two-stage "detect-then-classify" paradigm, decoupling localization from semantic interpretation.

- **Detector Architecture (YOLOv8n):** This nano-scale model utilizes a **CSPDarknet** backbone and **PAN-FPN** neck for efficient feature aggregation. Critically, it employs a **Decoupled Detection Head** (separating classification and regression tasks) and is optimized for latency with a total of **8.2 GFLOPs** .
- **Classifier Architecture (ResNet18):** Localization crops are passed to a ResNet18 model. This model leverages **transfer learning** from ImageNet; the final fully-connected layer is replaced and the **full network is fine-tuned end-to-end (all layers participate in backpropagation)** to shift the output from **512 → 1000** classes to **512 → 6** gesture classes.
  Residual connections are utilized to mitigate the vanishing gradient problem, allowing the network to learn high-level representations that are often superior to handcrafted features.



Example of full-frame image vs. hand crop (224x224) generated by the YOLOv8n detector with 10% padding. This illustrates the scale-invariant input used for the ResNet18 classifier

.

3. Experimental Results: Performance Benchmarking

Table 1: Model-Level Validation Metrics (Classification on pre-detected crops; Detection on full-frame validation set)

Metric	Classical (MediaPipe + SVM)	Deep Learning (YOLO + ResNet)
Validation Accuracy	91.65%	**99.69%**
Precision (Macro)	0.93	**0.994 (YOLO)**
Recall (Macro)	0.92	**0.989 (YOLO)**
F1 Score (Macro)	0.92	**0.99**
Inference Latency	**5–10 ms (CPU)**	15–30 ms (MPS/GPU)
Model Weight/Size	**~1 MB**	~68 MB (23MB YOLO / 45MB CNN)

Note on Validation Accuracies: The figures 91.65% (SVM) and 99.69% (CNN) were observed during model development on held-out validation sets. They represent model-level classification performance and must be reproduced on full-frame test images for authoritative end-to-end benchmarking.

Analysis of Detection Robustness:

- **Evaluation Warning:** Benchmarking on pre-cropped 224x224 datasets results in artificially low detection rates (~2-38%) compared to full-frame images. End-to-end evaluation must prioritize full-frame testing to reflect real-world performance.

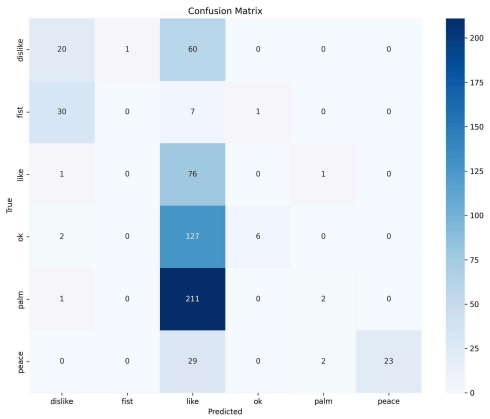
The YOLOv8n detector achieved an **mAP@0.5 = 0.995** and an **mAP@0.5:0.95 = 0.855** after 20 epochs of training. This high localization precision across varying IoU thresholds ensures that the subsequent CNN classifier receives highly accurate crops. While the classical approach is more resource-efficient, the deep learning pipeline provides a critical

7.8% accuracy gain

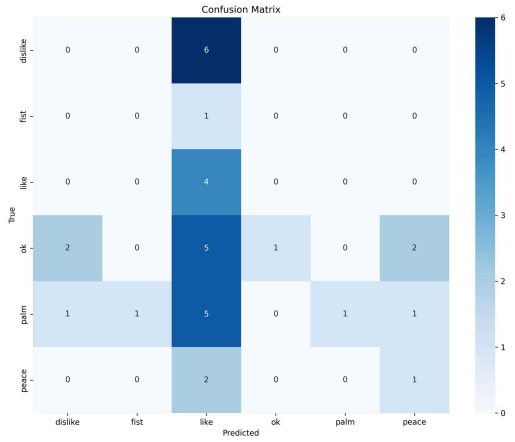
in gesture classification, making it the preferred choice for high-precision applications.

4. Failure Analysis: Constraints and Robustness Gaps

Failure Mode	Primary Cause	Pipeline
"Like" vs. "Palm" Confusion	Similar landmark configurations with thumb extended.	Classical (MediaPipe)
Occlusion Errors	Predicted landmarks for hidden joints are often invalid.	Classical (MediaPipe)
Missed Detections	Insufficient training epochs or small hand sizes.	Deep Learning (YOLO)
Crop Boundary Artifacts	The bounding box cuts off fingers near the edge.	Deep Learning (YOLO)



Confusion Matrix classical



Confusion Matrix deep

Mitigation and Implementation Strategies:

- **Spatial Padding:** Implement **10% padding** to crops to prevent finger "amputation" at the bounding box boundaries.
- **Data Leakage Prevention:** During dataset construction, samples were stratified by **user_id** rather than random image splits, ensuring the model generalizes to new hands rather than memorizing specific subjects.
- **Geometric Refinement:** The current feature vector includes PIP angles for the four fingers (index, middle, ring, pinky) but not the thumb. Future work could add thumb-specific angle or MCP-IP-TIP features to resolve 'like' vs. 'palm' ambiguities.

5. Ethical Considerations and Regulatory Framework

A Senior Research Lead must prioritize ethical readiness alongside technical performance.

- **Privacy and Local Processing:** The "local-first" architecture ensures that no biometric data or landmarks are transmitted to external servers, aligning with **GDPR** and **CCPA** compliance.
- **Bias and Fairness Audit:** The **HaGRID sample 30k source** dataset is predominantly Eastern European. Performance may degrade on underrepresented **skin tones** or **age-related morphology** (e.g., child-sized hands).
- **Environmental Footprint:** Transfer learning was used to reduce the carbon footprint. Fine-tuning was limited to **20 epochs for YOLO** and **up to 50 epochs for the CNN (early stopping with patience=5)** to minimize the energy consumption associated with model development.

6. Conclusion and Future Directions

The analysis confirms that the **Deep Learning pipeline** is superior for accuracy-critical tasks (**99.69% accuracy**), while the **Classical pipeline** is the optimal choice for edge deployment where latency and size (~1 MB) are the primary constraints.**Strategic Takeaways:**

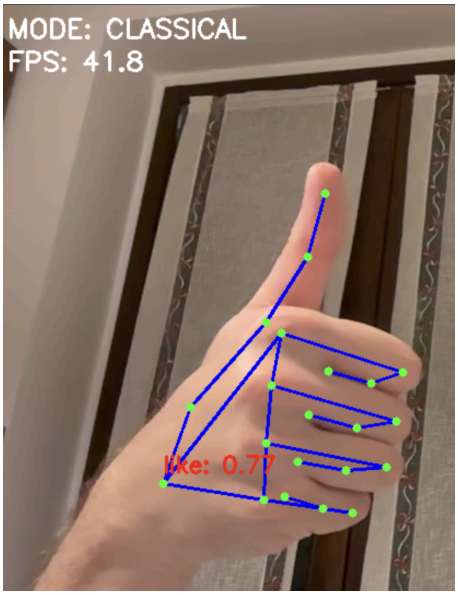
1. **Hardware Allocation:** Deploy the Classical pipeline for Edge/Mobile (CPU) and the Deep Learning pipeline for High-End GPU environments.
2. **Primary Long-Term Goal: Full Sign Language Recognition (ASL/LSF)**
    a. The current implementation is limited to 6 gestures due to dataset size and computational power constraints.
    The long-term objective of the project is to achieve full sign language recognition to serve as a comprehensive communication tool for non-verbal users.
3. **Temporal Smoothing:** Implement majority voting over sequences to stabilize predictions.
4. **Quantization:** Apply **INT8 Quantization** to the CNN for mobile optimization.
5. **Dataset Diversification:** Audit performance on multi-ethnic datasets to mitigate identified demographic biases.

7. References and Bibliography

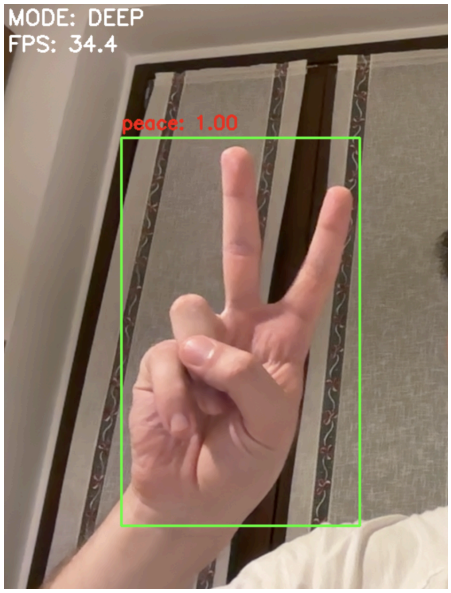
1. Kapitanov et al. "HaGRID — HAnd Gesture Recognition Image Dataset." https://github.com/hukenovs/hagrid
2. Google MediaPipe. "Hand Landmarker." https://developers.google.com/mediapipe/solutions/vision/hand_landmarker
3. Ultralytics. "YOLOv8." https://github.com/ultralytics/ultralytics
4. He, K., Zhang, X., Ren, S., & Sun, J. "Deep Residual Learning for Image Recognition." CVPR 2016.
5. scikit-learn. "Support Vector Machines." https://scikit-learn.org/stable/modules/svm.html
6. OpenCV. "Open Source Computer Vision Library." https://opencv.org/

8. Signature and Thanks

I would like to extend my **sincere gratitude** to the faculty for the opportunity to conduct this research.
The development of this **hand gesture detector**, designed as a dedicated alternative input for **non-verbal users**, represents a long-term pursuit fueled by technical curiosity and a commitment to **assistive technology**.



Live example of webcam classical



Live example of webcam deep

Thanks,
Lucian Claudiu Diaconu
Student | BSc in Computer Engineering & Artificial Intelligence

EPICODE
Institute of Technology

02/06/2026, Milan